

3D Curvilinear Grid Generation Package

User Manual and Algorithm Reference

May 13, 2026

Abstract

This document provides a comprehensive reference for the 3D curvilinear grid generation package. The package implements three grid generation methods—parabolic, hyperbolic, and elliptic—for constructing body-fitted structured meshes suitable for seismic wave numerical simulation with finite-difference methods. This manual covers the mathematical foundations, detailed algorithmic descriptions, grid quality metrics, post-processing capabilities, configuration guide, file formats, and usage instructions. All mathematical derivations and algorithmic descriptions are drawn directly from the C source code implementation.

Contents

1	Introduction	2
1.1	Background	2
1.2	Method Comparison	2
1.3	Coordinate System Convention	3
1.4	Software Architecture	3
2	Mathematical Foundations	4
2.1	Coordinate Transformation	4
2.2	Tangent Vectors	4
2.3	Metric Tensors	4
2.4	Jacobian Determinant	5
2.5	Orthogonality	5
2.6	Normal Vectors	5
2.7	Arc-Length Parameterization	6
3	Parabolic Grid Generation	7
3.1	Overview	7
3.2	Boundary Specification	7
3.3	Step Length Control	7
3.4	Predictor-Corrector Algorithm	8
3.4.1	Prediction Step	8
3.4.2	Correction Step	8
3.4.3	Thomas Algorithm	8
3.4.4	Boundary Conditions	9
3.5	Coordinate Permutation	9
3.6	Algorithm Summary	9
4	Hyperbolic Grid Generation	10
4.1	Overview	10
4.2	Governing Equations	10
4.2.1	Orthogonality Constraints	10
4.2.2	Volume Conservation	10
4.2.3	Linearized System	11
4.3	Block Tridiagonal Solver	11
4.3.1	Block Thomas Algorithm	11
4.4	Smoothing Coefficients	11
4.5	Boundary Conditions	12
4.5.1	Type 1: Floating Boundary (Neumann-like)	12
4.5.2	Type 2: Cartesian Boundary (Dirichlet-like)	12
4.6	Arc-Length Stretching	12
4.7	Algorithm Summary	13

5	Elliptic Grid Generation	14
5.1	Overview	14
5.2	Governing Equations	14
5.3	Boundary Specification	14
5.4	Transfinite Interpolation (TFI) Initialization	15
5.5	Source Terms	15
5.5.1	Dirichlet Boundary Source Terms	15
5.5.2	Hilgenstock Source Terms	15
5.5.3	Clustering Source Terms	15
5.6	SOR Iteration	16
5.6.1	Convergence Criteria	16
5.7	MPI Domain Decomposition	16
5.7.1	Topology	16
5.7.2	Ghost Cell Exchange	16
5.7.3	Global Convergence	16
5.7.4	Source Term Communication	17
5.8	Algorithm Summary	17
6	Grid Quality Metrics	18
6.1	Overview	18
6.2	Orthogonality Metrics	18
6.3	Jacobian Determinant	18
6.4	Smoothness Metrics	18
6.5	Step Size Metrics	19
6.6	Minimum Distance	19
7	Post-Processing	20
7.1	Grid Merging	20
7.2	Grid Sampling (Upsampling)	20
7.3	Arc-Length Stretching	20
7.4	PML Layer Support	21
8	Configuration Guide	22
8.1	JSON Configuration Format	22
8.2	Common Parameters	22
8.3	Parabolic-Specific Parameters	22
8.4	Hyperbolic-Specific Parameters	23
8.5	Elliptic-Specific Parameters	23
8.6	Post-Processing Parameters	23
9	Installation and Usage	24
9.1	Prerequisites	24
9.1.1	C Compilation	24
9.1.2	Python Environment	24
9.2	Building	24
9.3	Workflow	24
9.3.1	Stage 1: Generate Boundary Data (Python)	25
9.3.2	Stage 2: Run Grid Generation (C)	25
9.3.3	Stage 3: Post-Processing and Visualization	25

10 File Formats	26
10.1 Geometry File	26
10.1.1 6-Boundary Format (Elliptic)	26
10.1.2 2-Boundary Format (Parabolic)	26
10.1.3 1-Boundary Format (Hyperbolic)	27
10.2 Step File	27
10.3 Arc-Length File	27
10.4 NetCDF Output Format	28
10.4.1 Global Attributes	28
11 Mathematical Derivations	29
11.1 Derivation of the SOR Formula	29
11.2 Derivation of the Hyperbolic System	29

Chapter 1

Introduction

1.1 Background

Curvilinear grid generation is a fundamental pre-processing step for finite-difference seismic wave simulation in domains with complex topography and subsurface structures. Unlike Cartesian grids, curvilinear grids conform to irregular boundaries such as the free surface (topography), eliminating the staircase approximation errors that plague Cartesian methods at material interfaces.

In three dimensions, the computational domain is mapped from a unit cube $(\xi, \eta, \zeta) \in [0, 1]^3$ to physical space (x, y, z) . This mapping requires careful handling of three coordinate directions and six boundary surfaces, making the grid generation problem significantly more complex than its 2D counterpart.

The physical coordinates of grid points are denoted by:

$$\mathbf{r}(i, j, k) = (x_{i,j,k}, y_{i,j,k}, z_{i,j,k})^T, \quad i = 1, \dots, n_x, j = 1, \dots, n_y, k = 1, \dots, n_z \quad (1.1)$$

where n_x, n_y, n_z are the number of grid points in each direction.

1.2 Method Comparison

This package provides C implementations of three distinct grid generation methods:

Property	Parabolic	Hyperbolic	Elliptic
Boundary surfaces	2 (start + target)	1 (initial surface)	6 (all faces)
Generation mechanism	Predictor-corrector marching	Orthogonality + volume conservation marching	Elliptic PDE iteration
Computational cost	Low	Low	High (iterative)
Parallelization	None	None	MPI (3D decomposition)
Topography handling	Moderate	Extreme (cliffs, dams)	General
Grid quality control	Clustering coefficient	Smoothing + arc-stretch	Source terms P, Q, R
Output format	NetCDF (partitioned)	NetCDF (partitioned)	NetCDF (partitioned)

Table 1.1: Comparison of 3D grid generation methods.

1.3 Coordinate System Convention

Throughout this manual, computational coordinates are denoted by (ξ, η, ζ) corresponding to discrete indices (i, j, k) . The physical coordinate system is right-handed with x positive to the right, y positive forward, and z positive upward. The marching direction (for parabolic and hyperbolic methods) can be along any of the three computational axes.

1.4 Software Architecture

Component	Source Directory	Language
Boundary preparation	<code>*/example/prep_grid/</code>	Python 3
Boundary export	<code>plotting/export_c.py</code>	Python 3
Visualization	<code>plotting/</code>	Python 3 (Matplotlib)
Grid generation (elliptic)	<code>elliptic/src/</code>	C + MPI
Grid generation (hyperbolic)	<code>hyperbolic/src/</code>	C
Grid generation (parabolic)	<code>parabolic/src/</code>	C + MPI
Post-processing	<code>grid-post-process/src/</code>	C
Shared libraries	<code>lib/</code>	C

Figure 1.1: Software component organization.

Chapter 2

Mathematical Foundations

2.1 Coordinate Transformation

A 3D curvilinear grid defines a one-to-one mapping from computational coordinates (ξ, η, ζ) to physical coordinates (x, y, z) :

$$\mathbf{r}(\xi, \eta, \zeta) = \begin{pmatrix} x(\xi, \eta, \zeta) \\ y(\xi, \eta, \zeta) \\ z(\xi, \eta, \zeta) \end{pmatrix} \quad (2.1)$$

The grid generation problem is: given boundary surface positions in physical space, determine the interior mapping such that the resulting grid possesses desired properties (smoothness, orthogonality, appropriate cell volumes).

2.2 Tangent Vectors

The covariant tangent vectors at each grid point are computed using central finite differences for interior points and one-sided differences at boundaries:

$$\mathbf{r}_\xi(i, j, k) = \frac{\partial \mathbf{r}}{\partial \xi} \approx \frac{\mathbf{r}(i+1, j, k) - \mathbf{r}(i, j, k)}{\Delta \xi} \quad (2.2)$$

$$\mathbf{r}_\eta(i, j, k) = \frac{\partial \mathbf{r}}{\partial \eta} \approx \frac{\mathbf{r}(i, j+1, k) - \mathbf{r}(i, j, k)}{\Delta \eta} \quad (2.3)$$

$$\mathbf{r}_\zeta(i, j, k) = \frac{\partial \mathbf{r}}{\partial \zeta} \approx \frac{\mathbf{r}(i, j, k+1) - \mathbf{r}(i, j, k)}{\Delta \zeta} \quad (2.4)$$

In practice, $\Delta \xi = \Delta \eta = \Delta \zeta = 1$ is used since the computational coordinates are discrete indices.

2.3 Metric Tensors

The covariant metric tensor components g_{ij} form a 3×3 symmetric positive-definite matrix:

$$g_{11} = \mathbf{r}_\xi \cdot \mathbf{r}_\xi, \quad g_{22} = \mathbf{r}_\eta \cdot \mathbf{r}_\eta, \quad g_{33} = \mathbf{r}_\zeta \cdot \mathbf{r}_\zeta \quad (2.5)$$

$$g_{12} = \mathbf{r}_\xi \cdot \mathbf{r}_\eta, \quad g_{13} = \mathbf{r}_\xi \cdot \mathbf{r}_\zeta, \quad g_{23} = \mathbf{r}_\eta \cdot \mathbf{r}_\zeta \quad (2.6)$$

The derived metric coefficients used throughout the code are:

$$\alpha_1 = g_{22}g_{33} - g_{23}^2 \quad (2.7)$$

$$\alpha_2 = g_{11}g_{33} - g_{13}^2 \quad (2.8)$$

$$\alpha_3 = g_{11}g_{22} - g_{12}^2 \quad (2.9)$$

And the cross-coupling terms:

$$\beta_{12} = g_{13}g_{23} - g_{12}g_{33} \quad (2.10)$$

$$\beta_{23} = g_{12}g_{13} - g_{11}g_{23} \quad (2.11)$$

$$\beta_{13} = g_{12}g_{23} - g_{13}g_{22} \quad (2.12)$$

2.4 Jacobian Determinant

The Jacobian determinant (cell volume) is computed as the scalar triple product of the three tangent vectors:

$$J = \mathbf{r}_\xi \cdot (\mathbf{r}_\eta \times \mathbf{r}_\zeta) \quad (2.13)$$

Explicitly in component form:

$$J = \begin{vmatrix} x_\xi & x_\eta & x_\zeta \\ y_\xi & y_\eta & y_\zeta \\ z_\xi & z_\eta & z_\zeta \end{vmatrix} \quad (2.14)$$

A valid grid requires $J > 0$ at every grid point. A negative Jacobian indicates cell folding (crossing grid lines), which renders the grid unusable for numerical simulation.

2.5 Orthogonality

Grid line orthogonality is described by the angles between pairs of coordinate directions:

$$\theta_{\xi\eta} = \arccos\left(\frac{g_{12}}{\sqrt{g_{11}g_{22}}}\right) \times \frac{180}{\pi} \quad (2.15)$$

$$\theta_{\xi\zeta} = \arccos\left(\frac{g_{13}}{\sqrt{g_{11}g_{33}}}\right) \times \frac{180}{\pi} \quad (2.16)$$

$$\theta_{\eta\zeta} = \arccos\left(\frac{g_{23}}{\sqrt{g_{22}g_{33}}}\right) \times \frac{180}{\pi} \quad (2.17)$$

Ideal orthogonality requires all three angles to be 90° , meaning $g_{12} = g_{13} = g_{23} = 0$.

2.6 Normal Vectors

The unit normal vector to a surface spanned by $\mathbf{r}_\xi$ and $\mathbf{r}_\eta$ is:

$$\mathbf{n} = \frac{\mathbf{r}_\xi \times \mathbf{r}_\eta}{|\mathbf{r}_\xi \times \mathbf{r}_\eta|} \quad (2.18)$$

This normal vector is used extensively in the parabolic predictor step for extrapolation from the current grid layer.

2.7 Arc-Length Parameterization

The normalized arc length along a coordinate direction provides a measure of cumulative distance:

$$s(i) = \frac{\sum_{k=1}^i \Delta s(k)}{\sum_{k=1}^n \Delta s(k)}, \quad \Delta s(k) = |\mathbf{r}(k) - \mathbf{r}(k-1)| \quad (2.19)$$

This is used for grid point redistribution (arc-length stretching) to control clustering near boundaries or regions of interest.

Chapter 3

Parabolic Grid Generation

3.1 Overview

The parabolic method generates 3D grids by a predictor-corrector marching scheme from one boundary surface (the starting surface) toward another (the target surface). The method can march along any of the three coordinate directions (x, y, or z) by appropriate permutation of coordinates. The method is computationally efficient and suitable for moderate topography where the marching direction can be reasonably aligned with the coordinate transformation.

3.2 Boundary Specification

For z-direction marching (most common), two boundary surfaces are required:

- $\mathbf{r}_{bz1}(i, j)$ — bottom boundary surface, shape $(n_x, n_y, 3)$
- $\mathbf{r}_{bz2}(i, j)$ — top boundary surface, shape $(n_x, n_y, 3)$

For x-direction marching, the boundaries are on the y - z planes:

- $\mathbf{r}_{bx1}(k, j)$ — left boundary surface, shape $(n_z, n_y, 3)$
- $\mathbf{r}_{bx2}(k, j)$ — right boundary surface, shape $(n_z, n_y, 3)$

For y-direction marching:

- $\mathbf{r}_{by1}(k, i)$ — front boundary surface, shape $(n_z, n_x, 3)$
- $\mathbf{r}_{by2}(k, i)$ — rear boundary surface, shape $(n_z, n_x, 3)$

3.3 Step Length Control

The spacing between grid layers in the marching direction is controlled by a normalized step distribution $\bar{s}(k)$, where $k = 1, \dots, n_z - 1$. The step values are normalized to sum to 1:

$$\bar{s}(k) = \frac{s(k)}{\sum_{j=1}^{n_z-1} s(j)} \quad (3.1)$$

Smaller step values produce clustering of grid layers near the starting boundary, while larger values spread them out.

3.4 Predictor-Corrector Algorithm

3.4.1 Prediction Step

For each point at layer k , the predictor estimates the position at layer $k + 1$ using a weighted combination of two estimates:

1. **Normal direction point:** Extrapolates from layer k along the unit surface normal:

$$\mathbf{r}_{\text{norm}} = \mathbf{r}_k + \hat{c}_1 \cdot R \cdot \mathbf{n}_k \quad (3.2)$$

where $\mathbf{n}_k$ is the surface unit normal computed from $\mathbf{r}_\xi \times \mathbf{r}_\eta$, R is the local step length, and $\hat{c}_1$ is a curvature parameter.

2. **Linear interpolation point:** Interpolates linearly between the current layer and the target surface:

$$\mathbf{r}_{\text{lin}} = \mathbf{r}_k + \hat{c}_1 \cdot (\mathbf{r}_{\text{top}} - \mathbf{r}_k) \quad (3.3)$$

The final prediction is a blending of these two estimates with an exponential weight controlled by the clustering coefficient c :

$$\mathbf{r}_{k+1}^{\text{pred}} = e^{-c \cdot \hat{t}} \cdot \mathbf{r}_{\text{norm}} + (1 - e^{-c \cdot \hat{t}}) \cdot \mathbf{r}_{\text{lin}} \quad (3.4)$$

where $\hat{t}$ is the normalized marching coordinate at layer k . The coefficient c controls how quickly the grid transitions from surface-following (normal) behavior near the starting surface to linear behavior near the target surface. Larger values of c produce more clustering near the starting boundary.

3.4.2 Correction Step

The correction step solves a tridiagonal system of equations derived from the elliptic grid generation equations to smooth the predicted grid along each computational plane. For each ζ (marching direction) layer, a one-dimensional smoothing operator is applied along the ξ and η directions:

$$\alpha_1 \mathbf{r}_{i-1,j}^{\text{new}} + \alpha_2 \mathbf{r}_{i,j}^{\text{new}} + \alpha_3 \mathbf{r}_{i+1,j}^{\text{new}} = \mathbf{r}_{i,j}^{\text{pred}} \quad (3.5)$$

This tridiagonal system is solved using the Thomas algorithm (a simplified form of Gaussian elimination specialized for tridiagonal matrices), which has $O(n)$ computational complexity.

3.4.3 Thomas Algorithm

For a tridiagonal system $a_i x_{i-1} + b_i x_i + c_i x_{i+1} = d_i$:

1. **Forward elimination:**

$$c'_1 = c_1/b_1 \quad (3.6)$$

$$c'_i = c_i/(b_i - a_i c'_{i-1}), \quad i = 2, \dots, n-1 \quad (3.7)$$

$$d'_1 = d_1/b_1 \quad (3.8)$$

$$d'_i = (d_i - a_i d'_{i-1})/(b_i - a_i c'_{i-1}), \quad i = 2, \dots, n \quad (3.9)$$

2. **Backward substitution:**

$$x_n = d'_n \quad (3.10)$$

$$x_i = d'_i - c'_i x_{i+1}, \quad i = n-1, \dots, 1 \quad (3.11)$$

3.4.4 Boundary Conditions

At geometric symmetry boundaries, linear extrapolation is used:

$$\mathbf{r}_0 = 2\mathbf{r}_1 - \mathbf{r}_2 \quad (3.12)$$

This ensures that grid lines approach the boundary smoothly without artificial curvature.

3.5 Coordinate Permutation

After grid generation with a non-z marching direction, the coordinate axes are permuted to restore the standard (x, y, z) ordering. For example, if marching is in the x-direction, the generated coordinates are permuted from (y, z, x) back to (x, y, z) .

3.6 Algorithm Summary

1. Read boundary surfaces and step distribution
2. Set grid points on the starting boundary ($k = 1$)
3. For each marching step $k = 2, \dots, n_z$:
 - (a) For each (i, j) on the current layer, predict $\mathbf{r}_{i,j,k}$ using normal extrapolation blended with linear target
 - (b) Correct the predicted points by solving tridiagonal systems along ξ and η directions
 - (c) Apply boundary conditions
4. Permute coordinates if needed
5. Compute quality metrics and export to NetCDF

Chapter 4

Hyperbolic Grid Generation

4.1 Overview

The hyperbolic grid generation method constructs 3D grids by marching outward from a single initial boundary surface while simultaneously enforcing orthogonality constraints and volume conservation (or equivalently, a specified grid spacing distribution). This method was originally proposed by Steger and Chaussee (1980) and later extended to 3D by Chan and Steger (1992). It is particularly well-suited for domains with extreme topography (steep cliffs, near-vertical structures) where parabolic or elliptic methods may fail to maintain grid quality.

The hyperbolic method's key advantages are:

- Requires only one boundary surface (not two or six)
- Computationally efficient (marching, not iterative)
- Naturally enforces orthogonality at the starting surface
- Robust for extreme topography

4.2 Governing Equations

Let $\mathbf{r}(\xi, \eta, \zeta)$ be the grid mapping, where ζ is the marching direction. The grid lines must satisfy two fundamental constraints at each layer:

4.2.1 Orthogonality Constraints

Grid lines in the marching direction (ζ) must be orthogonal to the two tangential directions (ξ and η):

$$\mathbf{r}_\xi \cdot \mathbf{r}_\zeta = 0 \tag{4.1}$$

$$\mathbf{r}_\eta \cdot \mathbf{r}_\zeta = 0 \tag{4.2}$$

These conditions ensure that the grid is locally orthogonal at every interior point, which is essential for minimizing numerical errors in finite-difference wave propagation.

4.2.2 Volume Conservation

The third equation comes from specifying the cell volume (Jacobian determinant) according to a prescribed distribution:

$$\mathbf{r}_\xi \cdot (\mathbf{r}_\eta \times \mathbf{r}_\zeta) = V(\xi, \eta, \zeta) \tag{4.3}$$

or equivalently, controlling the grid spacing along the marching direction:

$$|\mathbf{r}_\zeta| = \Delta s(\xi, \eta, \zeta) \quad (4.4)$$

where Δs is derived from the step distribution file.

4.2.3 Linearized System

Expanding $\mathbf{r}_\zeta$ in terms of the known grid at layer k and the unknown at layer $k + 1$:

$$\mathbf{r}_\zeta \approx \mathbf{r}_{k+1} - \mathbf{r}_k \quad (4.5)$$

Substituting into the orthogonality and volume constraints yields a linear system that can be solved for $\mathbf{r}_{k+1}$.

4.3 Block Tridiagonal Solver

The resulting linear system has a block tridiagonal structure with 3×3 blocks, reflecting the three spatial coordinates at each grid point. The general form is:

$$B_i \mathbf{r}_{i-1,k+1} + C_i \mathbf{r}_{i,k+1} + D_i \mathbf{r}_{i+1,k+1} = E_i \quad (4.6)$$

where B_i, C_i, D_i are 3×3 matrices and $\mathbf{r}_i$ is the 3-component grid position vector.

4.3.1 Block Thomas Algorithm

The block Thomas algorithm generalizes the scalar Thomas algorithm:

1. **Forward elimination** ($i = 1, \dots, n$):

$$G_1 = C_1^{-1} D_1 \quad (4.7)$$

$$G_i = (C_i - B_i G_{i-1})^{-1} D_i, \quad i = 2, \dots, n-1 \quad (4.8)$$

$$F_1 = C_1^{-1} E_1 \quad (4.9)$$

$$F_i = (C_i - B_i G_{i-1})^{-1} (E_i - B_i F_{i-1}), \quad i = 2, \dots, n \quad (4.10)$$

2. **Backward substitution** ($i = n, \dots, 1$):

$$\mathbf{r}_n = F_n \quad (4.11)$$

$$\mathbf{r}_i = F_i - G_i \mathbf{r}_{i+1}, \quad i = n-1, \dots, 1 \quad (4.12)$$

The 3×3 matrix inversions in the forward elimination are computed using Cramer's rule with explicit determinant formulas for efficiency.

4.4 Smoothing Coefficients

To prevent grid line crossing and improve smoothness, a dissipation (smoothing) term is added. The smoothing coefficient at each point depends on several factors:

1. **Marching distance factor:**

$$S = \sqrt{\frac{k}{n_z - 1}} \quad (4.13)$$

Grid points farther from the starting surface receive stronger smoothing.

2. **Angular dependence:** The angle α between grid lines affects smoothing:

$$c_\alpha = \begin{cases} \frac{1}{1 - \cos^2 \alpha}, & \cos \alpha \geq 0 \\ 1, & \cos \alpha < 0 \end{cases} \quad (4.14)$$

3. **Grid density factor:** The ratio of grid points in the ξ and η directions:

$$N_\xi = \frac{n_\xi}{n_\xi + n_\eta} \quad (4.15)$$

4. **Combined coefficient:**

$$c_{\text{smooth}} = c_0 \cdot N_\xi \cdot S \cdot \Delta \xi_{\text{mod}} \cdot c_\alpha \quad (4.16)$$

where c_0 is the user-specified base smoothing coefficient.

4.5 Boundary Conditions

The hyperbolic method supports two types of boundary conditions at the edges of each computational plane:

4.5.1 Type 1: Floating Boundary (Neumann-like)

Boundary points adjust freely, damped by the epsilon parameter. For the boundary at $\xi = 0$:

$$\mathbf{r}_0^{n+1} = \mathbf{r}_0^n + (1 + \varepsilon)(\mathbf{r}_1^{n+1} - \mathbf{r}_1^n) - \varepsilon(\mathbf{r}_2^{n+1} - \mathbf{r}_2^n) \quad (4.17)$$

where ε (typically 0.01–0.1) controls how tightly the boundary follows the interior. Smaller ε produces a more tightly coupled boundary that moves with the interior; larger ε allows the boundary to drift more independently.

4.5.2 Type 2: Cartesian Boundary (Dirichlet-like)

One coordinate is fixed to maintain a Cartesian boundary condition:

$$x_0^{n+1} = x_0^n \quad (4.18)$$

$$y_0^{n+1} = y_0^n + (y_1^{n+1} - y_1^n) \quad (4.19)$$

$$z_0^{n+1} = z_0^n + (z_1^{n+1} - z_1^n) \quad (4.20)$$

This is useful when the boundary must remain on a specific plane (e.g., symmetry planes or domain boundaries that must stay aligned with the Cartesian axes).

4.6 Arc-Length Stretching

An optional arc-length stretching step can redistribute grid points along the marching direction to cluster points near boundaries where higher resolution is needed. The redistribution follows a user-provided normalized arc-length function $s(k) \in [0, 1]$:

$$\mathbf{r}_k^{\text{new}} = \mathbf{r}(\xi, \eta, s(k)) \quad (4.21)$$

Points are interpolated from the uniformly-spaced grid to positions specified by the arc-length distribution.

4.7 Algorithm Summary

1. Read the initial boundary surface and step distribution
2. Optionally read the arc-length stretching file
3. Set grid points on the initial surface ($k = 1$)
4. For each marching step $k = 2, \dots, n_z$:
 - (a) Compute tangent vectors $\mathbf{r}_\xi, \mathbf{r}_\eta$ from layer $k - 1$
 - (b) Compute unit normal vector $\mathbf{n} = (\mathbf{r}_\xi \times \mathbf{r}_\eta) / |\mathbf{r}_\xi \times \mathbf{r}_\eta|$
 - (c) Construct block tridiagonal system from orthogonality and volume constraints
 - (d) Add smoothing (dissipation) terms
 - (e) Solve block tridiagonal system via block Thomas algorithm
 - (f) Apply boundary conditions (Type 1 or Type 2)
5. Optionally apply arc-length stretching in the marching direction
6. Compute quality metrics and export to NetCDF

Chapter 5

Elliptic Grid Generation

5.1 Overview

The elliptic method generates grids by solving a system of Poisson-type elliptic partial differential equations. Unlike marching methods, the elliptic method simultaneously determines all interior grid points through iterative solution, providing the most general and controllable grid generation approach. All six boundary surfaces are specified, and the interior is determined by the elliptic PDE system with user-controlled source terms.

The method supports MPI parallel execution via 3D Cartesian domain decomposition, making it suitable for large-scale grid generation problems.

5.2 Governing Equations

The elliptic grid generation system is based on the Poisson equations for the computational coordinates in physical space:

$$\xi_{xx} + \xi_{yy} + \xi_{zz} = P(\xi, \eta, \zeta) \quad (5.1)$$

$$\eta_{xx} + \eta_{yy} + \eta_{zz} = Q(\xi, \eta, \zeta) \quad (5.2)$$

$$\zeta_{xx} + \zeta_{yy} + \zeta_{zz} = R(\xi, \eta, \zeta) \quad (5.3)$$

where P , Q , R are source terms that control grid point clustering and orthogonality. Inverting these equations to solve for physical coordinates in computational space yields:

$$\sum_{i=1}^3 \sum_{j=1}^3 g^{ij} \mathbf{r}_{\xi_i \xi_j} + P \mathbf{r}_\xi + Q \mathbf{r}_\eta + R \mathbf{r}_\zeta = 0 \quad (5.4)$$

where g^{ij} are the contravariant metric tensor components.

5.3 Boundary Specification

The elliptic method requires all six boundary surfaces:

- bx_1, bx_2 : boundaries at $\xi = 0$ and $\xi = 1$, shape $(n_y, n_z, 3)$
- by_1, by_2 : boundaries at $\eta = 0$ and $\eta = 1$, shape $(n_x, n_z, 3)$
- bz_1, bz_2 : boundaries at $\zeta = 0$ and $\zeta = 1$, shape $(n_x, n_y, 3)$

All six surfaces must be specified consistently so that edges and corners match.

5.4 Transfinite Interpolation (TFI) Initialization

An initial grid is generated by transfinite interpolation (TFI) from the six boundary surfaces. TFI provides a smooth initial guess that exactly matches all boundaries, significantly reducing the number of SOR iterations required for convergence.

The TFI formula for a 3D domain is:

$$\begin{aligned}
\mathbf{r}_{\text{TFI}}(\xi, \eta, \zeta) = & (1 - \xi)\mathbf{r}_{bx1}(\eta, \zeta) + \xi\mathbf{r}_{bx2}(\eta, \zeta) \\
& + (1 - \eta)\mathbf{r}_{by1}(\xi, \zeta) + \eta\mathbf{r}_{by2}(\xi, \zeta) \\
& + (1 - \zeta)\mathbf{r}_{bz1}(\xi, \eta) + \zeta\mathbf{r}_{bz2}(\xi, \eta) \\
& - (1 - \xi)(1 - \eta)\mathbf{r}_{\text{edge}_1} - \xi(1 - \eta)\mathbf{r}_{\text{edge}_2} - \dots \quad (\text{edge corrections}) \\
& + \text{corner correction terms}
\end{aligned} \tag{5.5}$$

The edge and corner correction terms subtract the double-counted and triple-counted contributions from the 1D and 2D interpolants.

5.5 Source Terms

5.5.1 Dirichlet Boundary Source Terms

For the Dirichlet formulation (`diri_gene`), source terms P , Q , R are computed on the boundary surfaces to enforce orthogonality and specified grid point spacing near the boundaries. On a boundary face, ghost points are projected along the surface normal:

$$\mathbf{r}_{\text{ghost}} = \mathbf{r}_{\text{bound}} + \mathbf{r}_{\xi} \tag{5.6}$$

where $\mathbf{r}_{\xi}$ at the boundary is the surface tangent direction projected normally. The source terms are then computed from the difference between the ghost point projection and the actual interior point.

5.5.2 Hilgenstock Source Terms

The Hilgenstock formulation (`higen_gene`) provides adaptive source terms based on angle and spacing deviation from targets:

$$\Delta\theta = \frac{\theta_0 - \theta}{\theta_0}, \quad \theta_0 = \frac{\pi}{2} \tag{5.7}$$

$$P = a \cdot \tanh(\Delta\theta_{\xi}) \tag{5.8}$$

$$Q = a \cdot \tanh(\Delta\theta_{\eta}) \tag{5.9}$$

$$R = a \cdot \tanh(\Delta\theta_{\zeta}) \tag{5.10}$$

where $a = 0.2$ is a relaxation constant. The tanh function provides smooth saturation so that large deviations do not cause instability.

5.5.3 Clustering Source Terms

Additional source terms control grid clustering toward specific boundaries using exponential decay functions:

$$P(\xi, \eta, \zeta) = \sum_{i=1}^4 a_i \cdot \text{sign}(\xi) \cdot e^{-d_i|\xi - \xi_{\text{ref}}|} \tag{5.11}$$

where a_i are user-specified amplitude coefficients and d_i are decay weights. These terms attract grid lines toward the specified reference surfaces.

5.6 SOR Iteration

The discretized Poisson system is solved using Successive Over-Relaxation (SOR) with $\omega = 1.0$ (Gauss-Seidel in practice). At each interior point (i, j, k) , the updated position is:

$$\mathbf{r}_{i,j,k}^{\text{new}} = (1 - \omega)\mathbf{r}_{i,j,k}^{\text{old}} + \frac{\omega}{2(\alpha_1 + \alpha_2 + \alpha_3)} \begin{bmatrix} \alpha_1(\mathbf{r}_{i+1,j,k} + \mathbf{r}_{i-1,j,k}) \\ +\alpha_2(\mathbf{r}_{i,j+1,k} + \mathbf{r}_{i,j-1,k}) \\ +\alpha_3(\mathbf{r}_{i,j,k+1} + \mathbf{r}_{i,j,k-1}) \\ +2\beta_{12}\mathbf{r}_{ij} + 2\beta_{23}\mathbf{r}_{jk} + 2\beta_{13}\mathbf{r}_{ik} \\ +\alpha_1 P\mathbf{r}_\xi + \alpha_2 Q\mathbf{r}_\eta + \alpha_3 R\mathbf{r}_\zeta \end{bmatrix} \quad (5.12)$$

5.6.1 Convergence Criteria

Convergence is measured by the maximum relative change across all interior points:

$$\max_{i,j,k} \left(\frac{|\Delta x|}{\Delta x_i}, \frac{|\Delta y|}{\Delta y_j}, \frac{|\Delta z|}{\Delta z_k} \right) < \epsilon \quad (5.13)$$

where ϵ is the convergence tolerance (`iter_err`, typically 10^{-4} to 10^{-6}). Iteration stops when either convergence is achieved or the maximum number of iterations (`max_iter`) is reached.

5.7 MPI Domain Decomposition

5.7.1 Topology

The 3D grid is decomposed using a Cartesian process topology with $p_x \times p_y \times p_z$ MPI processes. Each process owns a contiguous subdomain of the global grid and handles both interior updates and boundary communication.

5.7.2 Ghost Cell Exchange

Each process maintains a halo (ghost) layer around its subdomain. After each SOR iteration, processes exchange boundary data with their six neighbors using non-blocking MPI send/receive operations:

- East/West neighbors: exchange y - z planes
- North/South neighbors: exchange x - z planes
- Top/Bottom neighbors: exchange x - y planes

Processes at the global domain boundaries have `MPI_PROC_NULL` neighbors, simplifying the communication logic.

5.7.3 Global Convergence

Convergence requires a global reduction across all MPI processes. Each process computes its local maximum change, and the global maximum is obtained via `MPI_Allreduce` with `MPI_MAX`:

```
1 double local_max_res = compute_local_max_residual();
2 double global_max_res;
3 MPI_Allreduce(&local_max_res, &global_max_res, 1,
4               MPI_DOUBLE, MPI_MAX, MPI_COMM_WORLD);
```

5.7.4 Source Term Communication

Source terms computed on boundary faces are shared with neighboring processes using `MPI_Allreduce` with `MPI_SUM`, accumulating contributions from all overlapping subdomains.

5.8 Algorithm Summary

1. Read six boundary surfaces
2. Create MPI Cartesian communicator with $p_x \times p_y \times p_z$ topology
3. Compute TFI initial grid
4. Compute source terms P , Q , R on boundaries
5. **Iterate until convergence:**
 - (a) Update interior points using SOR formula
 - (b) Exchange ghost cells with MPI neighbors
 - (c) Update source terms at boundaries (Hilgenstock only)
 - (d) Check convergence via global reduction
6. Compute quality metrics
7. Each process exports its subdomain to a partitioned NetCDF file

Chapter 6

Grid Quality Metrics

6.1 Overview

The package computes grid quality metrics at every grid point during generation. These metrics enable quantitative assessment of grid suitability for seismic simulation.

6.2 Orthogonality Metrics

Three orthogonality metrics measure the deviation of grid line intersections from 90°:

$$\text{orth_xiet}(i, j, k) = 90 - |\theta_{\xi\eta}(i, j, k) - 90| \quad (6.1)$$

$$\text{orth_xizt}(i, j, k) = 90 - |\theta_{\xi\zeta}(i, j, k) - 90| \quad (6.2)$$

$$\text{orth_etzt}(i, j, k) = 90 - |\theta_{\eta\zeta}(i, j, k) - 90| \quad (6.3)$$

where $\theta_{\xi\eta}$ is computed from the metric tensors as:

$$\theta_{\xi\eta} = \arccos\left(\frac{g_{12}}{\sqrt{g_{11} \cdot g_{22}}}\right) \times \frac{180^\circ}{\pi} \quad (6.4)$$

A perfect grid has all three orthogonality metrics equal to 90. The metric decreases as grid lines deviate from perpendicular. A value of 0 indicates that the two grid directions are parallel (worst case). In practice, values above 70 are considered acceptable, and values above 85 are excellent.

6.3 Jacobian Determinant

The Jacobian determinant measures the signed volume of each grid cell:

$$\text{jacobi}(i, j, k) = \mathbf{r}_\xi \cdot (\mathbf{r}_\eta \times \mathbf{r}_\zeta) \quad (6.5)$$

A positive Jacobian ($J > 0$) indicates a valid right-handed cell. A negative Jacobian indicates cell folding (negative volume), which renders the grid unusable. All grid generation methods monitor the minimum Jacobian to detect failures.

6.4 Smoothness Metrics

Grid smoothness is quantified by the ratio of adjacent cell sizes along each coordinate direction:

$$\text{smooth_xi}(i, j, k) = \max\left(\frac{|\mathbf{r}_{i+1} - \mathbf{r}_i|}{|\mathbf{r}_i - \mathbf{r}_{i-1}|}, \frac{|\mathbf{r}_i - \mathbf{r}_{i-1}|}{|\mathbf{r}_{i+1} - \mathbf{r}_i|}\right) \quad (6.6)$$

A perfectly smooth grid has a smoothness value of 1.0 (uniform spacing). Values significantly larger than 1 indicate abrupt changes in grid spacing, which can cause numerical dispersion and instability in wave propagation simulations.

For boundary points where a two-sided difference is not available, a one-sided ratio using only the nearest interior neighbor is used instead.

6.5 Step Size Metrics

The physical step lengths between adjacent grid points in each direction:

$$\text{step_xi}(i, j, k) = |\mathbf{r}(i+1, j, k) - \mathbf{r}(i, j, k)| \quad (6.7)$$

$$\text{step_et}(i, j, k) = |\mathbf{r}(i, j+1, k) - \mathbf{r}(i, j, k)| \quad (6.8)$$

$$\text{step_zt}(i, j, k) = |\mathbf{r}(i, j, k+1) - \mathbf{r}(i, j, k)| \quad (6.9)$$

These are simple Euclidean distances and are useful for verifying that grid spacing satisfies the CFL stability condition and the points-per-wavelength requirement of seismic simulations.

6.6 Minimum Distance

An additional metric identifies the minimum distance from each grid point to the faces of adjacent cells using the point-to-plane distance formula:

$$d = \frac{|(\mathbf{r} - \mathbf{r}_0) \cdot \mathbf{n}|}{|\mathbf{n}|} \quad (6.10)$$

where $\mathbf{r}_0$ is a point on the face and $\mathbf{n}$ is the face normal vector. The global minimum of this metric indicates the most critically small cell face, which governs the maximum stable time step.

Chapter 7

Post-Processing

7.1 Grid Merging

Multiple independently generated grids can be merged to form a larger composite grid. Merging is supported along any of the three coordinate directions:

$$n_x^{\text{merged}} = n_x^{(1)} + n_x^{(2)} - 1 \quad (\text{x-direction merge}) \quad (7.1)$$

The shared boundary points are aligned and one set of duplicate coordinates is removed.

7.2 Grid Sampling (Upsampling)

Grids can be upsampled by integer factors in each direction using trilinear interpolation. Given a grid with dimensions (n_x, n_y, n_z) and upsampling factors (s_x, s_y, s_z) , the refined grid has dimensions:

$$(n'_x, n'_y, n'_z) = (s_x \cdot n_x, s_y \cdot n_y, s_z \cdot n_z) \quad (7.2)$$

For each new point, the 8 surrounding coarse-grid points are identified, and the interpolated value is:

$$f(x, y, z) = \sum_{i=0}^1 \sum_{j=0}^1 \sum_{k=0}^1 f_{ijk} \cdot w_x^i \cdot w_y^j \cdot w_z^k \quad (7.3)$$

where $w_x = (x - x_0)/(x_1 - x_0)$ is the normalized interpolation weight in the x direction, and similarly for y and z.

7.3 Arc-Length Stretching

Arc-length stretching redistributes grid points along a specified direction based on a normalized cumulative arc-length distribution $s \in [0, 1]$. This allows flexible clustering of grid points:

1. A step distribution $s(k)$ is specified, defining the relative spacing between layers
2. The cumulative arc-length function is computed: $L(k) = \sum_{i=1}^k s(i)$
3. Points are redistributed so that equal increments of the computational coordinate correspond to equal arc-length increments

The stretching can be applied in any of the three coordinate directions independently.

7.4 PML Layer Support

For seismic wave simulation with absorbing boundary conditions, the grid can be extended with Perfectly Matched Layer (PML) regions. The post-processor handles PML grid extension by:

1. Extending the grid point count to include PML layers at domain boundaries
2. Doubling grid points in PML regions for sufficient wave sampling
3. Balancing PML point distribution across MPI processes for load balance

Chapter 8

Configuration Guide

8.1 JSON Configuration Format

All runtime parameters are specified in a JSON configuration file. The parser uses the cJSON library with support for nested objects. Keys prefixed with # or // are treated as comments.

8.2 Common Parameters

Key	Type	Description
number_of_grid_points_x	int	n_x : grid points in ξ direction
number_of_grid_points_y	int	n_y : grid points in η direction
number_of_grid_points_z	int	n_z : grid points in ζ direction
geometry_input_file	string	Path to boundary geometry file
grid_export_dir	string	Output directory for grid NetCDF files
check_orth_xiet	int (0/1)	Compute ξ - η orthogonality metric
check_orth_xizt	int (0/1)	Compute ξ - ζ orthogonality metric
check_orth_etzt	int (0/1)	Compute η - ζ orthogonality metric
check_jac	int (0/1)	Compute Jacobian determinant
check_smooth_xi	int (0/1)	Compute ξ smoothness
check_smooth_et	int (0/1)	Compute η smoothness
check_smooth_zt	int (0/1)	Compute ζ smoothness
check_step_xi	int (0/1)	Compute ξ step size
check_step_et	int (0/1)	Compute η step size
check_step_zt	int (0/1)	Compute ζ step size

8.3 Parabolic-Specific Parameters

Key	Type	Description
step_input_file	string	Path to step distribution file
coef	int	Clustering coefficient (typical: 1–10). Larger values cluster more points near the starting boundary
t2b	int (0/1)	Top-to-bottom marching flag: 0 = start→target, 1 = target→start
parabolic.direction	string	Marching direction: "x", "y", or "z"

8.4 Hyperbolic-Specific Parameters

Key	Type	Description
<code>step_input_file</code>	string	Path to step distribution file
<code>flag_stretch</code>	int (0/1)	Enable arc-length stretching (requires arc-length file)
<code>hyp_arc_file</code>	string	Path to arc-length file (if <code>flag_stretch</code> = 1)
<code>hyperbolic.coef</code>	int	Base smoothing coefficient (typical: 1–5)
<code>hyperbolic.bdry_type</code>	[int, int]	Boundary types for ξ and η edges: 1 = floating, 2 = Cartesian
<code>hyperbolic.epsilon</code>	[float, float]	Damping parameters for floating boundaries (typical: 0.01–0.1)
<code>hyperbolic.direction</code>	string	Marching direction: "x", "y", or "z"

8.5 Elliptic-Specific Parameters

Key	Type	Description
<code>number_of_mpioprocs_x</code>	int	p_x : MPI processes along ξ
<code>number_of_mpioprocs_y</code>	int	p_y : MPI processes along η
<code>number_of_mpioprocs_z</code>	int	p_z : MPI processes along ζ
<code>grid.method.elli_diri.coef</code>	[int×4]	Source term amplitudes $a_1, \dots, a_4$
<code>grid.method.elli_diri.weight</code>	[float×2]	Exponential decay weights d_1, d_2
<code>grid.method.elli_diri.iter_err</code>	float	SOR convergence tolerance (typical: 10^{-5})
<code>grid.method.elli_diri.max_iter</code>	int	Maximum SOR iterations (typical: 5000–50000)

8.6 Post-Processing Parameters

Key	Type	Description
<code>input_grid_number</code>	int	Number of input grids to merge (for merge mode)
<code>merge_direction</code>	string	Merging axis: "x", "y", or "z"
<code>flag_sample</code>	int (0/1)	Enable grid upsampling
<code>sample_factor</code>	[int, int, int]	Upsampling factors $[s_x, s_y, s_z]$
<code>pml_weight_2x</code>	int (0/1)	Double point count for PML regions

Chapter 9

Installation and Usage

9.1 Prerequisites

9.1.1 C Compilation

- GCC ≥ 7.0 (for hyperbolic and post-processing)
- MPI compiler, e.g., mpicc (for parabolic and elliptic)
- NetCDF-C library (≥ 4.0) with development headers
- cJSON library (bundled in `lib/`)

9.1.2 Python Environment

- Python ≥ 3.10
- NumPy, SciPy, Matplotlib
- netCDF4 (Python binding to NetCDF-C)

9.2 Building

Set environment variables for library paths, then build each module:

```
1 # Set environment (adjust paths for your system)
2 export GNUHOME=/usr
3 export NETCDF=/path/to/netcdf
4
5 # Build hyperbolic (serial)
6 cd hyperbolic && make && cd ..
7
8 # Build parabolic (MPI)
9 cd parabolic && make && cd ..
10
11 # Build grid post-process (serial)
12 cd grid-post-process && make && cd ..
13
14 # Build elliptic (MPI)
15 cd elliptic && make && cd ..
```

9.3 Workflow

A typical grid generation workflow consists of three stages:

9.3.1 Stage 1: Generate Boundary Data (Python)

```
1 # Elliptic: 6-boundary model with Gaussian topography
2 cd elliptic/example/prep_grid
3 python3 creat_3d_gauss_boundary.py
4 # => ../data_file_3d.txt
5
6 # Hyperbolic: 1-boundary model
7 cd hyperbolic/example/prep_grid
8 python3 model1.py
9 # => ../data_file_3d.txt, ../step_file_3d.txt (via creat_step.py)
10
11 # Parabolic: 2-boundary model
12 cd parabolic/example/prep_grid
13 python3 model1.py
14 # => ../data_file_3d.txt, ../step_file_3d.txt (via creat_step.py)
```

Each boundary script includes a `flag_printf` variable. When set to 1, the script automatically generates and saves diagnostic plots to the `../plot/` directory.

9.3.2 Stage 2: Run Grid Generation (C)

```
1 cd elliptic/example
2 bash grid_generate.sh
3 # => output/coord_px*_py*_pz*.nc
```

The grid generation binary reads the JSON configuration file and the boundary geometry file, computes the grid, and writes NetCDF output.

9.3.3 Stage 3: Post-Processing and Visualization

```
1 # Post-process (merge, sample, stretch, PML)
2 cd grid-post-process/example
3 bash grid_post_proc.sh
4
5 # Visualize
6 cd plotting
7 python3 draw_grid.py
8 python3 draw_quality.py
```

Chapter 10

File Formats

10.1 Geometry File

The geometry file uses a C-format text representation parsed by `fscanf`. Comment lines begin with `#`.

10.1.1 6-Boundary Format (Elliptic)

Used when all six boundary surfaces are specified:

```
1 # nx number
2 101
3 # ny number
4 71
5 # nz number
6 51
7 # bx1 coords
8 0.000000000e+00 0.000000000e+00 -5.000000000e+02
9 ... (ny*nz = 71*51 lines)
10 # bx2 coords
11 ... (ny*nz lines)
12 # by1 coords
13 ... (nx*nz = 101*51 lines)
14 # by2 coords
15 ... (nx*nz lines)
16 # bz1 coords
17 ... (nx*ny = 101*71 lines)
18 # bz2 coords
19 ... (nx*ny lines)
```

10.1.2 2-Boundary Format (Parabolic)

Used when two boundary surfaces (start and target) are specified:

```
1 # nx ny number
2 401 301
3 # bz1 coords
4 ... (nx*ny lines)
5 # bz2 coords
6 ... (nx*ny lines)
```

10.1.3 1-Boundary Format (Hyperbolic)

Used when a single initial boundary surface is specified. The format varies by marching direction:

Z-direction marching:

```
1 # nx number
2 401
3 # ny number
4 401
5 # bz coords
6 ... (nx*ny = 401*401 lines)
```

X-direction marching:

```
1 # ny number
2 NY
3 # nz number
4 NZ
5 # bx coords
6 ... (ny*nz lines)
```

Y-direction marching:

```
1 # nx number
2 NX
3 # nz number
4 NZ
5 # by coords
6 ... (nx*nz lines)
```

10.2 Step File

Defines the relative spacing between grid layers in the marching direction:

```
1 # number of step
2 200
3 # step
4 -15.0
5 -15.0
6 ... (N lines)
```

Values are normalized internally. Negative values indicate decreasing spacing moving away from the starting surface.

10.3 Arc-Length File

Defines the cumulative arc-length distribution for grid point redistribution:

```
1 # number of points
2 201
3 # arc_len
4 0.000000
5 0.005000
6 ... (N lines, monotonically increasing from 0 to 1)
```

10.4 NetCDF Output Format

Grid coordinates and quality metrics are stored in NetCDF format with MPI-partitioned files:

- **Coordinate files:** `coord_px{i}_py{j}_pz{k}.nc`
 - Variables: `x`, `y`, `z` (float arrays, dimension sizes matching subdomain)
- **Quality metric files:** `{metric}_px{i}_py{j}_pz{k}.nc`
 - Variables correspond to quality metric names

10.4.1 Global Attributes

Each NetCDF file contains:

- **global_index_of_first_physical_points:** Array $[x_{\text{start}}, y_{\text{start}}, z_{\text{start}}]$ — 0-based starting indices in the global grid
- **count_of_physical_points:** Array $[n_x^{\text{local}}, n_y^{\text{local}}, n_z^{\text{local}}]$ — number of points in this subdomain (including ghost points)

These attributes enable reconstruction of the full global grid from the partitioned files.

Chapter 11

Mathematical Derivations

11.1 Derivation of the SOR Formula

Starting from the Poisson equations in computational space (Eqs. 5.1–5.3), we transform to physical space. The Laplacian in computational coordinates is:

$$\nabla^2 \mathbf{r} = \sum_{i=1}^3 \sum_{j=1}^3 g^{ij} \frac{\partial^2 \mathbf{r}}{\partial \xi_i^2} \xi_j + (\nabla^2 \xi) \frac{\partial \mathbf{r}}{\partial \xi} + (\nabla^2 \eta) \frac{\partial \mathbf{r}}{\partial \eta} + (\nabla^2 \zeta) \frac{\partial \mathbf{r}}{\partial \zeta} \quad (11.1)$$

Setting $\nabla^2 \xi = P$, $\nabla^2 \eta = Q$, $\nabla^2 \zeta = R$ gives the governing PDE.

Discretizing with central differences in computational space ($\Delta \xi = \Delta \eta = \Delta \zeta = 1$) and solving for $\mathbf{r}_{i,j,k}$ yields the SOR update formula in Section 5.4.

11.2 Derivation of the Hyperbolic System

Following Chan & Steger (1992), we linearize the orthogonality and volume constraints about the known grid at layer k . Let $\mathbf{r}^{k+1} = \mathbf{r}^k + \Delta \mathbf{r}$. Then:

$$\mathbf{r}_\xi \cdot (\mathbf{r}^{k+1} - \mathbf{r}^k)_\zeta = -\mathbf{r}_\xi \cdot \mathbf{r}_\zeta^k \quad (11.2)$$

$$\mathbf{r}_\eta \cdot (\mathbf{r}^{k+1} - \mathbf{r}^k)_\zeta = -\mathbf{r}_\eta \cdot \mathbf{r}_\zeta^k \quad (11.3)$$

With finite differences $\mathbf{r}_\zeta^{k+1} \approx \mathbf{r}^{k+1} - \mathbf{r}^k$, this produces a linear system for $\mathbf{r}^{k+1}$ at each k .